Resumable Uploads

upload in parts, resume what's missing

Watch the 4-min video

In one line: split large uploads into numbered parts under one upload ID, send parts in parallel, retry only failed parts, and after any interruption ask the server what it already has and send only the rest (S3 multipart, Azure block blobs, GCS resumable uploads or the open tus protocol); then let a lifecycle rule clean up uploads nobody finished.

1 Back to zero at 90%

A user uploads a 5 GB video over hotel Wi-Fi at 20 Mbit/s, which takes about 36 minutes. At 90% the connection drops. With a single `PUT`, those 4.5 GB are gone and the next attempt starts from byte zero; on a flaky network the chance of 36 failure-free minutes is low, so the user gives up. The fix is to make progress **durable in small pieces**.

2 Multipart upload

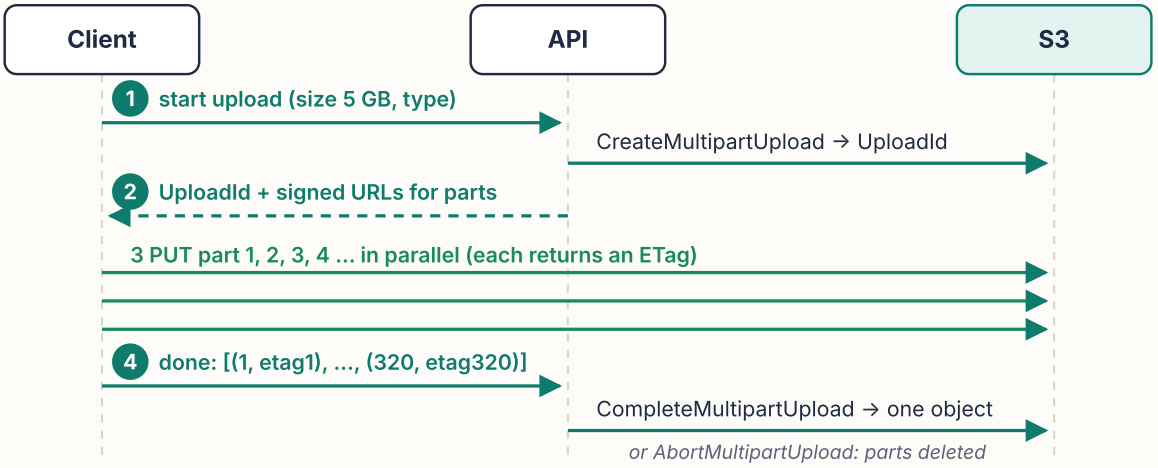


Figure 1. S3 multipart: one UploadId ties numbered parts together. Parts are invisible until Complete stitches them into one object.

1. **Create**: `CreateMultipartUpload` returns an `UploadId`.
2. **Upload parts**: `UploadPart` with `PartNumber` 1–10,000. Each part is 5 MiB to 5 GiB (only the last may be smaller) and returns an ETag. Parts can be sent in any order, in parallel, and re-sent (the last upload of a part number wins).
3. **Complete**: `CompleteMultipartUpload` with the list of (part number, ETag) in ascending order. S3 assembles the object atomically; until then nothing is visible under the key.
4. **Or abort**: `AbortMultipartUpload` deletes the parts. `ListParts` (1,000 per page) shows what has arrived.

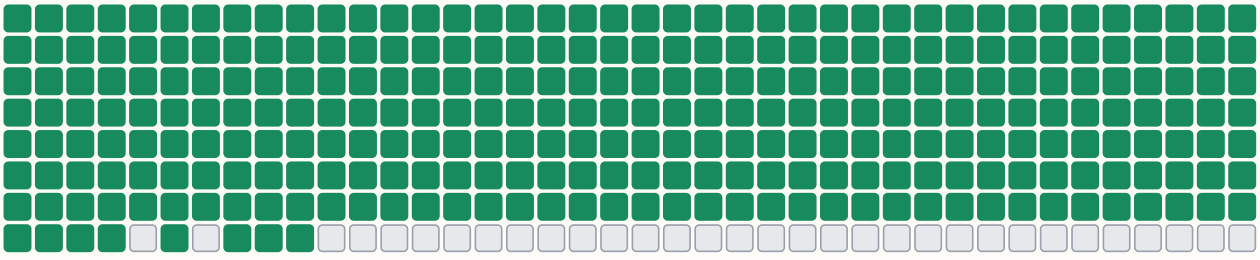
The browser never needs AWS credentials: the API makes the Create/Complete calls and pre-signs one URL per `UploadPart` (lesson 5). Choose the part size from the file size: with a 10,000-part cap, 16 MiB parts allow files up to ~160 GB; bigger files need bigger parts.

3 Two big wins: parallelism and small retries

- **Parallelism**: one TCP stream is often limited by latency and congestion control, not by link speed. Four to eight parts in flight usually fill a fast link; SDK transfer managers (boto3 `TransferConfig`, AWS CLI, Azure `max_concurrency`) do this for you.
- **Small retries**: a failed 16 MiB part is retried alone, with exponential backoff. The worst case after a drop is losing the parts in flight, not the file.
- **Integrity per part**: send a checksum with each part (`Content-MD5` or `x-amz-checksum-crc32` / `-sha256`); S3 rejects a corrupted part instead of storing it. Note the final ETag of a multipart object is *not* the MD5 of the file: it's a hash of the part hashes with a `-320` suffix.

Smaller parts mean less rework per failure but more requests (each `UploadPart` is a billed PUT) and more URLs to sign; 8–64 MiB is a common range.

4 Resumable: survive a crash or restart



4 parts were in flight when Wi-Fi dropped; parallel workers finish out of order, so gaps are normal

Figure 2. After reconnecting, the client asks what arrived and sends only the gaps; order doesn't matter until Complete.

Resuming needs state that outlives the connection. The **server** keeps the parts (keyed by UploadId); the **client** persists `{uploadId, key, partSize, file fingerprint}` in IndexedDB or the app's database. After a drop, a crash or even a reboot, the app reopens the same file, checks the fingerprint (name, size, modified time), asks the API for `ListParts`, gets fresh signed URLs for the missing part numbers (old ones may have expired) and continues. On mobile, OS background transfer services (e.g. iOS background `URLSession`) let parts keep uploading while the app is suspended.

Mechanism	Pieces	Ask what's there	Unfinished uploads
S3 multipart	parts 5 MiB–5 GiB, ≤ 10,000	`ListParts`	kept and billed until aborted; use a lifecycle rule
Azure block blob	`Put Block` with a block ID; ≤ 50,000 blocks; commit with `Put Block List`	`Get Block List` (uncommitted)	uncommitted blocks discarded after about a week
GCS resumable	byte ranges, chunks in multiples of 256 KiB	empty PUT with `Content-Range: bytes */total` → 308 + `Range`	session URI valid for one week
tus 1.0	byte offset; parallel via *concatenation* extension	`HEAD` → `Upload-Offset`	*expiration* extension; server deletes

5 The tus protocol

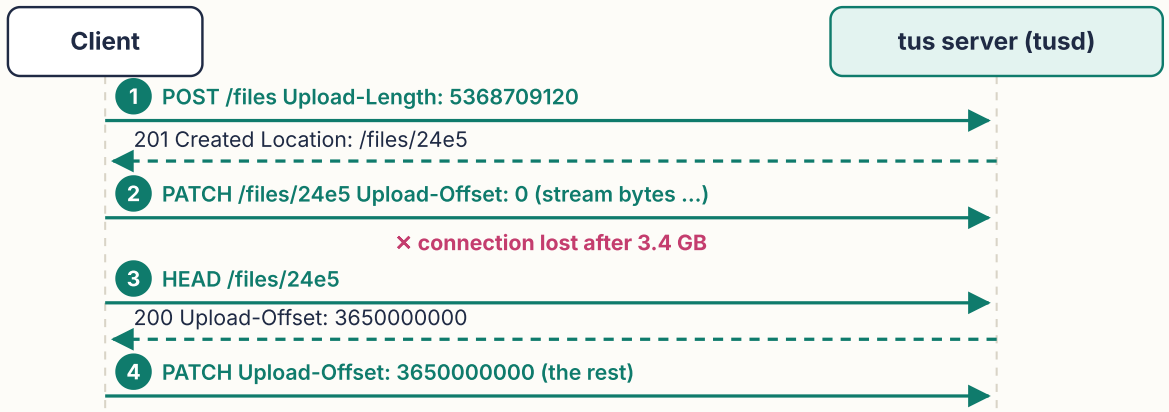


Figure 3. tus: create, PATCH bytes at an offset, and after a failure HEAD tells you where to continue. Every request carries `Tus-Resumable: 1.0.0`.

tus is an open HTTP protocol for resumable uploads. The core is two requests: `HEAD` returns the current `Upload-Offset`, and `PATCH` (body type `application/offset+octet-stream`) appends bytes starting at a given offset; the server rejects a PATCH whose offset doesn't match (409). Extensions add *creation* (POST with `Upload-Length`), *checksum*, *expiration*, *termination* (DELETE) and *concatenation* (upload pieces in parallel, then join). Clients: tus-js-client, Uppy, iOS and Android libraries; servers: tusd (which can store into S3, Azure or GCS), tus-node-server. Use tus when you want one protocol across web and mobile and your own server in the path for auth and hooks; use native multipart when you want bytes to go straight to the cloud provider.

6 Clean up abandoned uploads

An abandoned multipart upload keeps its parts: invisible in normal listings, but **billed**. A busy upload service can quietly accumulate terabytes of them. Add a lifecycle rule `AbortIncompleteMultipartUpload` with `DaysAfterInitiation: 3` (lesson 12), and set business limits: a maximum file size, a maximum number of parts and of concurrent uploads per user, and count in-progress uploads against quota, so nobody fills your storage on purpose.

7 Example: resuming a 5 GB upload

WORKED EXAMPLE: 320 PARTS, WI-FI DROPS AT PART 280

The file is 5,120 MiB; with 16 MiB parts that's **320 parts**. The app asks the API to start; the API calls `CreateMultipartUpload`, stores the UploadId against the `videos` row and returns signed URLs in batches of 50. The app uploads 4 parts at a time and saves progress locally. Around part 280 the Wi-Fi drops. Back online, the app calls `GET /uploads/{id}/parts`; the API's `ListParts` shows **288 of 320** present (in-flight parts finished out of order). The app requests fresh URLs for the 32 missing parts and sends them: 512 MiB, about 3½ minutes at 20 Mbit/s instead of 36 minutes from zero. Then it sends the 320 (part, ETag) pairs; the API calls `CompleteMultipartUpload` and the video appears as one object, triggering the processing pipeline (lesson 10).

TRADE-OFFS

- Multipart adds round trips and state; below ~100 MB a single PUT with a retry is simpler.
- Smaller parts: cheaper failures and finer progress, but more requests, signatures and per-request cost.
- Native multipart sends bytes straight to the cloud; tus is portable and uniform but puts your server (or tusd) in the data path.
- More parallelism helps fast links but hurts slow, shared ones (and phones on metered data).

COMMON MISTAKES

- No lifecycle rule for incomplete uploads: invisible parts pile up on the bill.
- Keeping the UploadId only in memory, so an app restart loses the ability to resume.
- Picking a part size that hits the 10,000-part cap for large files.
- Treating the multipart ETag as a file MD5 when verifying integrity.

8 Key terms

Multipart upload: object uploaded as numbered parts, then assembled

ListParts: call that returns the parts already received

tus: open HTTP protocol: POST to create, PATCH at offset, HEAD to resume

UploadId: identifier tying parts of one multipart upload together

Resumable upload: upload that can continue from its last durable point

AbortIncompleteMultipartUpload: lifecycle action that deletes unfinished uploads

9 Interview questions

Q1. How do you support resumable uploads? BASIC

Split the file into numbered parts under one upload ID, upload parts in parallel, retry only failed parts, and after a failure ask which parts exist and send only the rest, via multipart uploads or the tus protocol.

Q2. What about abandoned uploads? BASIC

A lifecycle rule aborts incomplete multipart uploads after a few days so their parts stop costing money; also cap file size and concurrent uploads per user.

Q3. Users upload 200 GB scientific files. What part size do you choose? SCENARIO

At least 200 GB ÷ 10,000 ≈ 20 MiB, so pick something like 32–64 MiB; that stays under the part cap and keeps the request count reasonable.

Q4. The browser tab is closed mid-upload. How does the user continue tomorrow? SCENARIO

Persist the upload ID, key, part size and a file fingerprint client-side; when the user re-selects the file, match the fingerprint, ListParts via the API, and upload only the missing parts with fresh URLs.

Q5. Upload throughput is poor on a 1 Gbit office link. What do you tune? SCENARIO

Raise concurrency (more parts in flight) and use reasonably large parts; one TCP stream rarely fills a long-latency fast link.

Q6. Why doesn't the ETag of a multipart object match the file's MD5? TRICKY

S3 computes it from the MD5s of the parts plus a part-count suffix; use per-part checksums or the additional checksum algorithms (e.g. SHA-256, CRC32C) for end-to-end verification.

Q7. Do parts have to be uploaded in order? TRICKY

No: any order, in parallel, and a part number can be re-uploaded; only Complete needs the part list in ascending order.

Next: SD-O-007 · File Metadata Storage